docker-control — User Manual

`docker-control` is a Rust CLI that manages Docker-based PHP/LAMP projects. It gives you a single, opinionated interface for the whole lifecycle of a project: scaffolding it from a template, running its container stack locally, wiring up SSH agent forwarding and a shared reverse proxy, and cutting releases and deploying them to remote servers.

It runs as a **standalone binary**. Every command below can be invoked two equivalent ways:

Form	Example
Standalone binary	`docker-control start`
Shorthand alias (if configured)	`dc2 start`

This manual uses `docker-control` throughout; substitute whichever form you prefer.

Table of contents

---

1. Installation

`docker-control` is distributed via Homebrew.

If you don't have Homebrew, install it first:

```
/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/
        Homebrew/install/HEAD/install.sh)"
```

Trust the tap. Modern Homebrew (with `HOMEBREW_REQUIRE_TAP_TRUST` set) refuses to load formulae from a non-official tap until you explicitly trust it:

```
brew trust INTERLIGENT-kommunzieren-GmbH/tap
```

Then install the tool (`brew install` taps the repository automatically):

```
brew install INTERLIGENT-kommunzieren-GmbH/tap/docker-control
```

This makes the `docker-control` command available.

> **Note:** if `brew install` fails with a trust error, run the `brew trust` command above first. Trusted entries are stored in `~/.homebrew/trust.json` (or under `$XDG_CONFIG_HOME/homebrew/`).

Building from source

The project is Rust 2024 edition. To build and test locally:

```
cargo build --release      # release binary in target/release/
        docker-control
cargo nextest run          # run the test suite (use nextest, not
        `cargo test`)
```

Installing dependencies

`docker-control` shells out to a number of external tools (see Dependencies). To install every Homebrew-installable dependency in one step — including optional ones:

```
docker-control install-deps
```

On startup, the tool also checks for missing dependencies and, on macOS / native Linux with Homebrew, offers to install the ones it can. Beyond that, individual commands install what they specifically need on demand — e.g. `deploy` will offer to install 7-Zip, and `start` the ACL tools — so you rarely need to run `install-deps` by hand. See Dependencies.

---

2. Core concepts

Managed project. Most commands require the current directory to be a *managed project*, marked by a `.managed-by-docker-control` sentinel file. `docker-control init` creates this file. Commands that mutate a project (`start`, `stop`, `build`, `console`, `setacl`, `update`, …) refuse to run outside a managed directory.

Two repositories. A project directory is the **infrastructure / operations** layer (the compose stack, config, secrets). The actual PHP **application lives in `htdocs/`, which is a separate git repository** mounted into the `php` container at `/var/www/html`. Release, merge, and deploy operate on the `htdocs/` repo (and optionally vendor modules under `htdocs/vendor/<name>/`).

Template-driven. New projects are scaffolded from an embedded template. The template is compiled into the binary and extracted to your OS config directory on first run (and whenever the version changes). Each project records which template it is synced to in `.docker-control/state.json`, so `docker-control update` can apply only what actually changed — and `start`/`status` can tell you when there *is* something to apply.

Ingress. An optional shared reverse-proxy stack that routes `https://` dev URLs to your project containers. It is a separate compose stack from your project.

---

3. Quick start

```
# 1. Create and enter a project directory
mkdir my-app && cd my-app

# 2. Scaffold the project (interactive: name, PHP version,
        optional git clone)
docker-control init

# 3. (Optional, once per machine) start the shared reverse proxy
docker-control start-ingress

# 4. Start the container stack
docker-control start

# 5. Trust the ingress CA (once per host) — only works after a
        project has been
#    started, since the CA is generated when the first domain is
        signed
docker-control trust-ca

# 6. Open a shell in the PHP container to run composer, bin/
        console, etc.
docker-control console

# 6. Check status at any time
docker-control status
```

Your app is then reachable at `https://<name>.lvh.me/` (the `lvh.me` domain resolves to `127.0.0.1`). phpMyAdmin is at `/_phpmyadmin/` and Mailpit at `/_mail/`.

---

4. Global options

These flags apply to (nearly) every command:

Flag	Description
`-d, --dir <DIRECTORY>`	Operate on the given project directory instead of the current one.
`--debug`	Enable verbose debug output.
`-h, --help`	Show help. At the top level (`docker-control --help`, `-h`, or

Flag	Description
	`help`) it prints a full project status summary and lists any custom commands. After a command (`docker-control <command> --help`) it prints that command's own help page — its arguments, options, and description.
`-V, --version`	Print the `docker-control` version. Top-level only — after a subcommand the flag belongs to that subcommand (`module link <m> --version <v>`) or, past a `--`, to the command being run in the container (`console -- php --version`).

SSH-agent daemon control flags (handled before normal command parsing):

Flag	Description
`--start-ssh-agent`	Start the SSH agent forwarding daemon.
`--stop-ssh-agent`	Stop the SSH agent forwarding daemon.
`--restart-ssh-agent`	Restart the SSH agent forwarding daemon.

See SSH agent forwarding — normally you don't need these because the daemon is started automatically.

Running `docker-control` with **no command** prints the project status and a hint to run `--help`.

---

5. Command reference

Project lifecycle

`init`

Initialize the current (or `--dir`) directory as a managed project from the template.

Interactively prompts for: - **Project name** (defaults to the directory name; sanitized to lowercase with separators replaced by _). - **PHP version** — one of `7.4`, `7.4-oci`, `8.2`, `8.2-oci`, `8.5`, `8.5-oci`. - Optionally **clone a git repository into `htdocs/`** (SSH URL recommended).

It copies the template, renames `.gitignore-dist` → `.gitignore`, creates the `htdocs/` directory, applies host ACLs, auto-selects a free database host port in the range `33060–33099`, and writes a `.env` file. If the directory is non-empty (and not already managed) it warns and asks for confirmation before overwriting.

```
docker-control init
```

`start`

Start the project containers in detached mode (`docker compose up -d`). Before starting, it does a throttled check (at most weekly) for outdated container images and offers to pull them. After starting, it re-applies host and container ACLs on `htdocs`.

On Linux this requires the ACL tools (`setfacl`/`getfacl`, from the `acl` formula); if they're missing, `start` offers to install them and aborts if they can't be obtained. macOS falls back to `chmod +a` and needs no extra tools. See Dependencies.

```
docker-control start
```

`stop`

Stop the project containers (`docker compose down`).

```
docker-control stop
```

`restart`

Stop then start the containers. Also runs the outdated-image check and re-applies ACLs. Like `start`, it requires the ACL tools on Linux.

```
docker-control restart
```

`status`

Show a detailed project status report — plugin management, git repository state (branch + uncommitted changes), deployment config and configured environments, and Docker container status — followed by `docker compose ps`.

```
docker-control status
```

`build [args...]`

Build the project containers. Any extra arguments are passed straight through to `docker compose build`.

```
docker-control build
docker-control build --no-cache
```

`pull`

Pull the latest images for the project (`docker compose pull`).

```
docker-control pull
```

Working inside the stack

`console [container] [-- command...]`

Open an interactive `bash` shell inside a container. Defaults to the `php` container, entered as the `www-data` user with the working directory at `/var/www/html`. Pass a service name to target a different container; pass `help` to list available services.

```
docker-control console          # php container as www-data
docker-control console db       # db container
docker-control console help     # list services
```

Run `composer`, `php`, `bin/console`, `artisan`, etc. **inside** the php container via `console`, since the application is mounted there. MariaDB is also reachable from the host at `127.0.0.1:${DB_HOST_PORT}`.

Everything after a `--` runs as a one-shot command instead of opening a shell, in the same container, as the same user, with the same working directory — and exits with the command's own status code, so it composes in scripts and `&&` chains:

```
docker-control console -- composer install
docker-control console -- php bin/console cache:clear
docker-control console db -- mysql -e 'show databases'
```

The `--` is required: without it the first argument is read as a service name, so `console ls -la` is an error rather than a command. It also means every flag after it belongs to the inner command — `console -- php --version` reports PHP's version, not docker-control's. The command is passed straight to the container as arguments — no shell is involved, so pipes, redirects, globs and `&&` need an explicit shell of their own:

```
docker-control console -- bash -lc 'ls -1 var/log/*.log | wc -l'
```

Output is piped verbatim when stdout isn't a terminal (`console -- cat composer.json > copy.json` writes the file unchanged), and a TTY is allocated when it is, so interactive commands such as `php -a` or a prompting `bin/console` generator still work.

`module <create|link|unlink|list>`

Check a vendor module out for local development. Vendor modules are installed from source, so `htdocs/vendor/<vendor>/<name>/` is a real git clone — but anything edited there is discarded by the next `composer install`. `module link` moves the clone to `htdocs/modules/<vendor>/<name>/` and adds a Composer `path` repository that symlinks it back into `vendor/`, so your edits survive and the module's git history stays usable (`release` and `merge` still find it through the symlink).

The path repository pins the version to whatever `composer.lock` already records for the package, via `options.versions`. Your `require` constraint is never touched, and there is no state file — the repository entry itself is the state.

The project containers must be running: Composer runs inside the `php` container.

Option	Description
`link [module]`	Link a module for development. Prompts when `module` is omitted.
`--version <VER>`	Pin a specific version instead of the one from `composer.lock`.
`unlink [module]`	Restore the module to a normal Composer install under `vendor/`.

Option	Description
`--purge`	Also delete the development checkout from `htdocs/modules/`.
`-y, --yes`	Skip the confirmation prompt when `--purge` finds unsaved work.
`list`	Show every vendor module and which are linked. Read-only; no containers needed.
`create <module>`	Scaffold a **new** module and link it, for a module that doesn't exist yet.
`--type <TYPE>`	Package type forwarded to `composer init` (default `library`).
`-y, --yes`	Run `composer init` with `--no-interaction` instead of asking its questions.

```
docker-control module list
docker-control module create acme/widget
docker-control module link acme/widget
docker-control module link acme/widget --version 2.4.x-dev
docker-control module link acme/widget -- -W --no-scripts   #
        extra composer update args
docker-control module unlink acme/widget
docker-control module unlink acme/widget --purge
```

`create` — *a module that doesn't exist yet*

`link` assumes the module is already installed. `create` is for starting one from nothing: it creates `htdocs/modules/<vendor>/<name>/` with a `src/` directory, runs `git init` on branch `main`, runs **`composer init` interactively** in the module directory so you answer Composer's usual questions, adds a PSR-4 autoload mapping for `src/` if `composer init` didn't, commits the scaffold, and then wires it into the application exactly as `link` would — a `path` repository pinned to `dev-main`, symlinked into `vendor/`.

```
docker-control module create acme/widget                #
        composer init asks its questions
docker-control module create acme/widget --yes          # accept
        composer init's defaults
docker-control module create acme/widget --type=symfony-bundle
```

The namespace is derived from the package name, so `acme/my-widget` autoloads `Acme\MyWidget\` from `src/`. The package name must be a valid, lowercase Composer name; an invalid one is rejected before anything is created.

`create` adds a `require`, and `link` never does. That is the one real difference between them: `link` works on a package the application already requires, whereas nothing requires a brand-new module, so `vendor/<vendor>/<name>` would never appear without it. This splits what you commit:

- the **`path` repository** must *not* be committed — `deploy` and `release` build from the committed tree, where `modules/` does not exist;
- the **`require`** *does* belong in a commit, but only once the module is pushed somewhere the application's other repositories can reach. Until then the application only installs on your machine.

Because of that, `unlink` refuses a module whose checkout has no git remote: it would remove the path repository — currently the package's only source — and leave the application requiring something nothing can supply. Push the module first, or drop it from the application with `docker-control console -- composer remove <vendor>/<name>`.

If `create` fails partway through (typically because Composer couldn't resolve the new package), the application side is rolled back — `composer.json`, `composer.lock` and the `.gitignore` entry — but **the new module is kept**. Those files are your work, not something Composer can reproduce.

`link` also adds `/modules/` to `htdocs/.gitignore` and registers the checkout as an additional git root in `.idea/vcs.xml`, so PhpStorm shows the module's branches, commits and diffs in the Git tool window. `unlink` removes the mapping again; the `.idea` handling is a no-op when the project has no `.idea` directory.

The `.gitignore` entry is written under a marker comment:

```
# docker-control: development module checkouts
/modules/
```

That marker is what makes the change reversible — it identifies the entry as the command's own, so `unlink` removes it (once nothing is linked and no checkout remains) while never touching a `/modules/` line your project already had. If your `.gitignore` already ignores `modules/`,

`link` adds nothing and `unlink` removes nothing. A link/unlink round trip therefore restores `composer.json`, `composer.lock`, `vendor/` and `.gitignore` exactly as they were.

> **Don't commit the link.** `composer.json` and `composer.lock` are tracked files, and `deploy/release` build from the committed tree — where `modules/` does not exist. Run `unlink` before committing, or keep the change out of your commits.

Two behaviours worth knowing: `unlink` uses `composer update --prefer-source`, which applies to the whole Composer run rather than just the named package, so anything else reinstalled in that run also becomes a source install. And after `unlink` swaps the symlink back for a real directory, PHP's realpath/opcache in the running container can serve stale paths — restart the `php` container if the application misbehaves.

`unlink` never destroys work: it leaves the development checkout in `htdocs/modules/` unless you pass `--purge`, and `--purge` checks for uncommitted changes and for commits that exist on no remote before asking to confirm.

`setacl`

Re-apply host and container ACL permissions on `htdocs` without restarting. Use this when host/container file permissions drift out of sync. The project containers must be running. On Linux it requires the ACL tools (`setfacl`/`getfacl`) and offers to install them if they're missing.

```
docker-control setacl
```

`doctor [--fix]`

Diagnose file-access problems for the project, and optionally repair them. The project containers must be running. It runs two checks:

- **Host ACL on `htdocs`** — whether the host user's rwX ACL (with inheritance) is actually applied, so you can edit files the container creates without `sudo`.
- **Container access to `/var/www`** — lists every path the container's `www-data` user cannot read or write. This surfaces the common case where Composer's home/XDG directories (`/var/www/.composer`, `.config`, `.cache`) were created later and slipped past the ACL, leaving Composer unable to write its home.

Without `--fix`, `doctor` only reports and exits non-zero if it finds problems (useful in scripts). With `--fix` it repairs both sides: it creates the Composer/XDG home directories (`.composer`, `.config`, `.cache`, `.local`), re-applies the container ACL (which also recomputes the ACL mask), and replays the host ACL on `htdocs`. Before touching the container ACL it verifies `setfacl` exists in the php image and, if the `acl` package is missing, tells you to update the image (`docker-control pull`) rather than failing with a cryptic error. `--fix` may prompt for `sudo` when re-applying the host ACL.

```
docker-control doctor          # report only
docker-control doctor --fix    # repair, then re-check
```

Ingress (reverse proxy)

The ingress is a shared reverse-proxy compose stack that serves your projects over HTTPS. You typically start it once per machine.

Command	Effect
`start-ingress`	Start the ingress containers (detached). Also syncs ingress volumes.
`stop-ingress`	Stop the ingress containers.
`restart-ingress`	Stop then start the ingress containers.
`status-ingress`	Show ingress container status.
`pull-ingress`	Pull the latest ingress images.
`trust-ca`	Trust the ingress CA certificate on this host.

`trust-ca`

Installs the ingress proxy's self-signed CA certificate into the host trust store so your project's `https://` dev URLs are trusted without browser warnings. It handles:

- **macOS** — the System keychain (via `sudo security add-trusted-cert`).
- **Linux / WSL** — the system anchor store (Debian/Ubuntu or RHEL layout).
- **Windows** — the Windows certificate store.
- **Browsers** — the NSS trust store used by Chrome/Chromium and Firefox (via `certutil`, from the Homebrew `nss` formula) on macOS/

Linux. `certutil` is only needed when a browser profile actually exists on the machine; when one does, it is **required** — `trust-ca` offers to install `nss` and errors out if it can't, rather than silently skipping the browser import. With no browser present, this step is quietly skipped.

The CA does not exist until the first domain is signed. The self-sign companion only generates the CA when it signs a project's first domain, so `start-ingress` alone is not enough — you must start at least one project first. If no CA is found, `trust-ca` tells you to start the ingress/a project first. Once a project is up, run `trust-ca` once per host. It may prompt for `sudo`/admin rights. Restart your browser afterward.

```
docker-control start-ingress   # start the shared proxy
docker-control start           # start a project (signs the first
        domain → CA is created)
docker-control trust-ca        # trust the now-existing CA
```

Release & deployment

`add-deploy-config`

Interactively add a deployment environment to `.deploy.json` (creating the file if needed). Prompts for environment name, SSH user, domain, server root, console command, description, optional Microsoft Teams webhook, optional COPS integration, and shared directories/files. See Configuration files.

```
docker-control add-deploy-config
```

> **Work in progress.** The `release` and `deploy` commands are still under active development and their behavior may change. When migrating a project, **Capistrano is currently used** for deployment rather than `docker-control deploy`.

`release [module]`

Create a new release with automatic semantic versioning. Optionally target a vendor module (otherwise you're prompted to choose the main project or a vendor module when modules exist). See Release & merge workflows in depth.

```
docker-control release
docker-control release some-vendor-module
```

`merge [module]`

Cherry-pick a release branch back into the primary branch via an isolated `<release>-merge` branch, skipping `release:`-prefixed commits, then push for a PR. See Release & merge workflows in depth.

```
docker-control merge
```

`deploy <env> [options]`

Deploy a selected release/tag to a configured environment. See Deployment in depth.

`deploy` compresses the release with `7z`, so 7-Zip is **required**: if it's missing, `deploy` offers to install it (Homebrew `p7zip`) and aborts up front if it can't, rather than failing partway through. See Dependencies.

Option	Description
`<env>`	Target environment name (must exist in `.deploy.json`). Required.
`-r, --release <REL>`	Deploy a specific release/tag, skipping interactive selection.
`--maintenance-mode <hard|soft>`	Maintenance mode to use with `--yes` (default `hard`).
`-y, --yes`	Skip all interactive prompts (non-interactive deploy).

```
docker-control deploy production
docker-control deploy staging --release 2.4.1
docker-control deploy production --yes --maintenance-mode soft
```

Custom commands

`create-control-script <name>`

Scaffold a new custom control script. Prompts for a description, whether the script should override a built-in command of the same name, and where to store it (inside `htdocs` or in the project root). See Custom control scripts.

```
docker-control create-control-script my-command
```

`<script-name> [args...]`

Any shell script under `control-scripts/` or `htdocs/.docker-control/control-scripts/` is dispatched as a subcommand. Custom commands appear in the top-level `--help` with their descriptions, and `docker-control <script-name> --help` shows the script's own help via its `_help_` hook (see Custom control scripts).

```
docker-control my-command arg1 arg2
docker-control my-command --help    # runs the script's _help_
        hook
```

Maintenance & housekeeping

`show-running`

List all running projects managed by `docker-control` across the machine (project name and directory), discovered via Docker container labels.

```
docker-control show-running
```

`update [--yes] [--check] [--force-template]`

Re-sync **this project** with the current template, applying only what actually changed.

`init`, `update` and `migrate` record the template's file hashes in `.docker-control/state.json` at the project root (outside `htdocs/`, and meant to be committed). That recorded state is a merge base, so each file can be classified exactly:

Situation	What `update` does
The template didn't change this file	nothing — your edits are irrelevant here
The template changed it, you didn't	applies it
New file in the template	adds it
You changed it and so did the template	asks: keep yours / take the template's / show the diff / write the template's copy as `*.dist`
File dropped from the template	reports it once; never deletes

Situation	What `update` does
No recorded state (project predates it)	asks once whether to review each differing file, take the template's version for all, or keep yours

Your answers are recorded, so "keep my version" means you aren't asked again — your edit now sits on top of the current template, and only a *further* upstream change to that file counts as a new conflict.

`secrets/*.txt` and `config/htpasswd` hold per-project values and are seeded at `init` only — `update` never overwrites them. `logs/` and `volumes/` are skipped. `.env` is never rewritten: when the template's `.env-dist` gains a key your `.env` lacks, the key is reported and you choose the value — and it keeps being reported until the key is actually there, since nothing else will add it for you. The project's own `.env-dist` (shipped by `init` as the reference list of available keys) is refreshed like any other template file. New `.gitignore-dist` entries are merged into `.gitignore`.

When it has anything to apply, `update` stops the project (if running), creates a timestamped `backup_<epoch>/`, applies the changes, and restarts the project. This **rewrites project files**, so it asks for confirmation; in a non-interactive context it refuses unless `--yes` is given. With `--yes`, a conflicting file is kept as-is and the template's version is written beside it as `*.dist` — an unattended run never discards local work.

Option	Description
`-y, --yes`	Skip the confirmation prompt (required for non-interactive use).
`--check`	Report pending changes and the diffs, change nothing, exit non-zero if anything is pending. Useful in CI.
`--force-template`	Overwrite every template-owned file, ignoring local modifications (`secrets/` and `config/htpasswd` are still preserved).

`start`, `restart` and `status` mention pending template changes automatically — and stay quiet when the template hasn't moved, so the notice only appears when there is something to do.

> `update` updates the *project*. To upgrade the *tool itself*, use `upgrade`.

```
docker-control update
docker-control update --check   # what would change? (exit 1 if
        anything)
docker-control update --yes     # non-interactive
```

`cleanup-backups [options]`

Remove old `backup_*` folders left behind by `update`/`migrate` (and clean up their PhpStorm exclude entries).

Option	Description
`-k, --keep <N>`	Keep the N most recent backups (default 5). Deletes the rest.
`--older-than <DAYS>`	Delete backups older than DAYS days. Mutually exclusive with `--keep`.
`--all`	Remove **all** backup folders. Mutually exclusive with `--keep` and `--older-than`.
`--dry-run`	List what would be deleted without deleting.
`-y, --yes`	Skip the confirmation prompt.

```
docker-control cleanup-backups --dry-run
docker-control cleanup-backups --keep 3
docker-control cleanup-backups --older-than 30 --yes
docker-control cleanup-backups --all          # remove every
        backup (prompts first)
```

Backups are deleted as the host user, without `sudo`. If a `backup_*` folder contains files owned by `root` (for example `vendor/` contents created inside the container), removal of that folder fails with a `Permission denied` warning and the command moves on to the next one — it never escalates privileges. When that happens, keep the host ACL on `htdocs` applied (see `doctor` / `setacl`) so container-created files stay host-owned, or remove the leftover folder manually.

Self-management

`install-deps`

Install every Homebrew-installable dependency (critical and optional) in one `brew install`. This command intentionally skips the normal dependency check so it can be run to fix missing dependencies. Requires Homebrew.

```
docker-control install-deps
```

`install-claude`

Install Claude Code and its codebase-memory-mcp companion using their official install scripts, then enable codebase-memory-mcp's auto-indexing of new projects. Each step runs its official installer directly (a `curl … | bash` one-liner, no extra confirmation) and streams the installer's own output. Requires `curl` and `bash`.

```
docker-control install-claude
```

`upgrade`

Upgrade `docker-control` itself via Homebrew (`brew upgrade …/docker-control`). The tool also performs a throttled (weekly) background check and, in an interactive terminal, may offer to upgrade when a newer version is available.

```
docker-control upgrade
```

`user-manual`

Open this manual (the bundled `USER-MANUAL.pdf`) in your default PDF application. The PDF ships with the Homebrew distribution; a copy is also embedded in the binary as a fallback. On **WSL** it opens the manual with the **Windows** default PDF viewer (via `wslpath` + `cmd.exe`/`explorer.exe`); on macOS it uses `open`, on other Linux `xdg-open`, and on Windows `start`. This command needs no external tools (not even Docker).

```
docker-control user-manual
```

---

6. Project layout

A scaffolded project looks like this:

Path	Purpose
`compose.yml`	The service stack definition. Template-owned.
`htdocs/`	**The application — a separate git repo** (gitignored here). Mounted at `/var/www/html`.
`htdocs/.docker-control/`	Per-project config: `.deploy.json`, `control-scripts/`, `deployment-scripts/<env>.rhai`.
`config/`	Container config: `apache-sites/`, `php.ini`, `mariadb.cnf`, `composer.config.json`, `crontab`, `htpasswd`, `ssmtp.conf`.
`secrets/`	DB credential files: `db_name.txt`, `db_user.txt`, `db_pw.txt`, `db_root_pw.txt`.
`volumes/`	Persistent data: `db/`, `valkey/`, `composer-cache/`, `db-share/`.
`logs/`	Container logs: `apache/`, `mariadb/`.
`releases/`	Git worktrees used during `release`/`merge`.
`deployments/`	Deployment archives and optional Rhai hook scripts.
`control-scripts/`	Project-local custom subcommands.
`.env`	Project settings (see below).
`.managed-by-docker-control`	Sentinel marking the directory as managed.

7. Configuration files

`.env`

Written by `init` and read at runtime (and by `compose.yml`).

Key	Meaning
`PROJECTNAME`	Project name; the compose project name and container label.
`BASE_DOMAIN`	Dev domain, e.g. `myapp.lvh.me` (resolves to `127.0.0.1`).
`ENVIRONMENT`	App environment, e.g. `development` (maps to `APP_ENV`).
`DB_HOST_PORT`	Host port mapped to MariaDB `3306` (auto-selected at init).
`PHP_VERSION`	Tag for the `fduarte42/docker-php:<PHP_VERSION>` image.
`XDEBUG_IP`	Host IP Xdebug connects back to (default `host.docker.internal`).
`IDE_KEY`	Xdebug IDE key.

Optional overrides consumed by `compose.yml` include `PHP_GC_MAX_LIFETIME`, `PHP_UPLOAD_LIMIT`, and `PHP_ERROR_REPORTING`.

`.deploy.json`

Deployment configuration. Search order:

1. `htdocs/.docker-control/.deploy.json` (preferred)
2. `.deploy.json` (project root, fallback)

The `version` field must be `"1.0"`. Example:

```
{
  "version": "1.0",
  "environments": {
    "production": {
      "user": "deploy",
      "domain": "production.example.com",
      "serviceRoot": "/var/www/html",
```

```
      "console_command": "bin/console",
      "description": "Production environment - stable releases
        only",
      "tags": ["production", "critical"],
      "teamsWebhookUrl": "https://outlook.office.com/webhook/...",
      "copsIntegration": false,
      "sharedDirectories": ["var/log", "public/uploads"],
      "sharedFiles": [".env.local"]
    },
    "staging": {
      "user": "deploy",
      "domain": "staging.projects.interligent.com",
      "serviceRoot": "/var/www/html",
      "description": "Staging environment for testing"
    }
  },
  "environmentOrder": ["production", "staging"],
  "defaults": {
    "serviceRoot": "/var/www/html",
    "domainSuffix": ".projects.interligent.com"
  }
}
```

Per-environment fields:

Field	Required	Meaning
`user`	yes	SSH user on the target server.
`domain`	yes	Target server hostname.
`serviceRoot`	no	Deploy root on the server (default `/var/www/html`).
`console_command`	no	Console entry point (default `bin/console`).
`description`	no	Human-readable description.
`tags`	no	Free-form labels.
`branch`	no	Associated branch (informational).
`teamsWebhookUrl`	no	

Field	Required	Meaning
		Microsoft Teams webhook for deploy notifications.
`copsIntegration`	no	Run COPS `cops:outdated` / `cops:permissions` during deploy.
`sharedDirectories`	no	Directories symlinked from `<serviceRoot>/shared/` into each release.
`sharedFiles`	no	Files symlinked from `<serviceRoot>/shared/` into each release.

The easiest way to create/extend this file is `docker-control add-deploy-config`.

8. The container stack

`compose.yml` defines the following services:

Service	Image	Role
`php`	`fduarte42/docker-php:${PHP_VERSION}-debug`	Apache + mod_php serving `htdocs/` at `/var/www/html`.
`db`	`mariadb:11.8`	Database; host port `${DB_HOST_PORT}` → `3306`.
`phpmyadmin`	`phpmyadmin`	DB admin UI at `/_phpmyadmin/`.
`mail`	`axllent/mailpit`	Catch-all mail UI at `/_mail/`.
`gotenberg`	`gotenberg/gotenberg:8-chromium`	PDF/document conversion service.
`redis`	`valkey/valkey:8.1`	

Service	Image	Role
		Redis-compatible cache/store.
logrotate	ghcr.io/fduarte42/logrotate	Rotates container logs.

Networking: the `frontend-tier` is the external `proxy` network (the ingress); the `backend-tier` is an internal bridge. DB credentials are provided via Docker `secrets` sourced from `secrets/*.txt`.

Prefer `docker-control` commands over raw `docker` / `docker compose`, because the CLI also wires up SSH agent forwarding, ACL fixes, ingress, secrets, and env handling.

---

9. SSH agent forwarding

Deploy and release run `composer install` inside Docker containers that need access to your SSH keys (e.g. to fetch private Composer packages). To make the host SSH agent available to containers, `docker-control` runs a small **forwarding daemon** that exposes the agent on TCP port `2222` (`SSH_AGENT_PORT`).

- The daemon is **started automatically** when missing (unless `DOCKER_CONTROL_SKIP_SSH_AGENT` is set, or the command doesn't need SSH). Once running, the tool publishes `SSH_AUTH_PORT=<bind_ip>:2222` for compose and deploy commands.
- The bind IP is platform-dependent (`127.0.0.1` on macOS/WSL/Docker Desktop; the `docker0` bridge IP on native Linux).
- The daemon PID/logs live at `/tmp/docker-control-ssh-agent.{pid,log,err}`.

Manual control:

```
docker-control --start-ssh-agent
docker-control --stop-ssh-agent
docker-control --restart-ssh-agent
```

If you see "SSH agent forwarding is not available", ensure `ssh-agent` is running and `SSH_AUTH_SOCK` is set in your shell.

---

10. Deployment in depth

> **Work in progress.** `deploy` is under active development and may change. When migrating a project, **Capistrano is currently used** for deployment instead.

`docker-control deploy <env>` runs, in order:

1. **Load config** for `<env>` from `.deploy.json`.
2. **Select a release** — fetches tags and prompts (or uses `--release`).
3. **Confirm** the deployment (skipped with `--yes`).
4. **Teams "started" notification** (if `teamsWebhookUrl` is configured).
5. **Build the archive** locally: check out the release tree with git, run `composer install -o` inside a `fduarte42/docker-php:<PHP_VERSION>` container (using SSH agent forwarding), then compress with `7z` into `deployments/<timestamp>_<release>.7z`.
6. **Transfer & extract** the archive to `<serviceRoot>/releases/<timestamp>_<release>/` over `scp`/`ssh`, then remove the archive.
7. **Prune** old releases, keeping the 5 most recent (never the live `current`).
8. **Shared paths** — symlink each `sharedDirectories`/`sharedFiles` entry from `<serviceRoot>/shared/` into the new release.
9. **Reload FPM** (`sudo php-fpm-reload.sh`).
10. **Maintenance mode** — enable `hard` or `soft` on the current and new releases.
11. **Hooks: `pre_deploy`** (Rhai; see below).
12. **Deployment tasks** — clear opcache, clear ORM metadata/query caches; optionally clear result cache, run `migrations:migrate`, and run `orm:schema-tool:update` (each prompted unless `--yes`).
13. **COPS integration** (if enabled) — `cops:outdated` and `cops:permissions`, with prompts to continue on failure.
14. **Hooks: `post_deploy`**.
15. **Manual pause** — in interactive mode, waits for ENTER so you can run ad-hoc commands on the server before going live.
16. **Flip the `current` symlink** to the new release.
17. **Disable maintenance mode** and do a final opcache clear.
18. **Hooks: `done_deploy`**.
19. **Teams "success"/"failed" notification**.

On failure the local archive is removed and (if configured) a "failed" Teams notification is sent; maintenance mode is best-effort restored.

Deployment hooks (Rhai)

You can inject custom SSH steps at three points: `pre_deploy`, `post_deploy`, and `done_deploy`. Hooks are Rhai scripts loaded from the first of:

1. `htdocs/.docker-control/deployment-scripts/<env>.rhai`
2. `deployments/scripts/<env>.rhai`

Each hook is an optional function receiving `(console_current, release_dir, console_new, server_root)` and can call `exec_ssh(command)` to run commands on the target server:

```
fn pre_deploy(console_current, release_dir, console_new,
server_root) {
    exec_ssh(console_new + " app:warmup");
}

fn post_deploy(console_current, release_dir, console_new,
server_root) {
    exec_ssh("echo Deployed " + release_dir);
}
```

Functions you don't define are simply skipped.

---

11. Release & merge workflows in depth

> **Work in progress.** The `release` workflow is still under active development and may change.

Both workflows operate on the `htdocs/` repository (or a vendor module under `htdocs/vendor/<name>/`) using isolated git **worktrees** under `releases/` so your working tree is never disturbed.

`release`

1. Optionally select the main project or a vendor module.
2. Pre-flight: `fetch`, determine the primary branch, update it.
3. Determine the version:
   - No release branches yet → initial release **`1.0.x`**.
   - **Breaking change?** → next major `X.0.x`.
   - **New feature?** → next minor `X.Y.x`.

    - Otherwise → **patch**: pick an existing `X.Y.x` branch, next patch `X.Y.Z`.
4. **Minor/major/initial** create a new `X.Y.x` **branch**: update `composer.json` version to `X.Y.x-dev`, generate `composer.lock` via Docker, commit both, generate a `CHANGELOG.md`, and push the branch.
5. **Patch** creates a **tag** on the `X.Y.x` branch: update `composer.json` to the exact version, generate the changelog, create tag `X.Y.Z`, and push the tag.

`release:`-prefixed commits are used for the composer.json/lock bookkeeping so they can be excluded when merging back.

`merge`

1. Optionally select the main project or a vendor module.
2. Pre-flight: `fetch`, determine primary branch, list release branches, pick one.
3. Create two worktrees: the release branch, and a new `<release>-merge` branch based on `origin/<primary>`.
4. Compute the commits on the release branch not in primary, **excluding `release:` commits**, and confirm.
5. Cherry-pick each commit. On conflict, choose to **launch the merge tool**, mark it **resolved**, or **abort**.
6. On success, push `<release>-merge` and print next steps for opening a PR from `<release>-merge` into the primary branch. The local merge branch is cleaned up on a successful push; on failure or if you decline the push, it's preserved for inspection.

---

12. Custom control scripts

Custom commands are plain `bash` scripts discovered from (in priority order):

1. `htdocs/.docker-control/control-scripts/`
2. `control-scripts/`

Create one with `docker-control create-control-script <name>`. It's run with the project directory as the working directory and `PROJECT_DIR` set in its environment.

A script supports three special probe arguments:

- `_desc_` — print a one-line description (shown in the top-level `--help` list).
- `_help_` — print a detailed help page (shown by `docker-control <name> --help`). If a script doesn't implement `_help_`, `--help` falls back to its `_desc_` description. The probe only runs on scripts that contain a quoted `_help_`, so older scripts are never executed just to render help.
- `_override_` — print `true` to always win a name clash with a built-in command.

Generated skeleton:

```
#!/bin/bash
set -e

if [[ "$1" == "_desc_" ]]; then
    # short description (shown in the command list)
    echo "My command description"
    exit 0
fi

if [[ "$1" == "_help_" ]]; then
    # detailed help (shown by `docker-control my-command --help`)
    echo "my-command - My command description"
    echo
    echo "Usage: docker control my-command [arguments]"
    exit 0
fi

# Optional: always override a same-named built-in command
if [[ "$1" == "_override_" ]]; then
    echo "true"
    exit 0
fi

echo "my-command - WAITING FOR IMPLEMENTATION"
exit 0
```

Name clashes

If a custom script has the same name as a built-in command (e.g. `build.sh`), you'll be prompted to choose which runs. Pressing Enter/Escape keeps the built-in. Add the `_override_` block to make your

script always win without a prompt. When a custom script wins a clash with a built-in that requires a managed project, the managed-project check is still enforced.

---

13. Environment variables

Variable	Effect
`DOCKER_CONTROL_SKIP_DEPENDENCY_CHECK`	Skip external tool dependency checks.
`DOCKER_CONTROL_SKIP_SSH_AGENT`	Skip SSH agent management.
`DOCKER_CONTROL_SKIP_IMAGE_CHECK`	Skip the outdated-image check on start/restart.
`DOCKER_CONTROL_SKIP_SELF_UPDATE_CHECK`	Skip the weekly self-update check.
`DOCKER_CONTROL_INGRESS_DIR`	Override the ingress directory location.
`DOCKER_CONTROL_TEMPLATE_DIR`	Override the template directory location.
`HOMEBREW_PREFIX`	Override the detected Homebrew prefix (used for ingress paths).
`SSH_AUTH_PORT`	Set automatically to `<bind_ip>:2222`; read by deploy/release Docker commands.
`SSH_AUTH_SOCK`	Standard SSH agent socket; required for agent forwarding.
`PHP_VERSION`	Read from the project `.env`; used for the `fduarte42/docker-php` image tag.

---

14. Dependencies

External tools `docker-control` relies on. Critical tools must be present or every command aborts at startup. The tools marked “on demand” below are not required in general, but are **mandatory for the specific command that uses them**: when you run such a command and its tool is missing, `docker-control` offers to install it there and then and aborts if it can’t, instead of failing later with a cryptic error. The remaining optional tools simply enable extra features.

Tool	Required	Used for
Docker (≥ 20.10)	yes	All container operations.
Docker Compose (≥ 2.4)	yes	Managing the service stack.
Git	yes	Release and merge workflows.
SSH	yes	Remote access during deploy.
SCP	yes	File transfers during deploy.
Bash	yes	Executing custom scripts.
Sudo	no	Elevated privileges (migration, `trust-ca`).
Rsync	no	Migration tasks.
7-Zip (`7z`, `p7zip` formula)	on demand	Building deployment packages — required by `deploy`.
setfacl / getfacl (`acl` formula)	on demand (Linux)	Host/container ACLs — required by `start`/`restart`/`setacl` on Linux (macOS falls back to `chmod`).
certutil (`nss` formula)	on demand	Browser CA trust for `trust-ca` — required when a browser NSS store exists.

Installing dependencies. Missing tools that have a Homebrew formula can be installed interactively when a command needs one, or all at once with `install-deps`. Because `docker-control` is distributed via Homebrew, **Homebrew itself is treated as non-optional**: if a tool needs installing but `brew` is not found, that's a hard error pointing you at https://brew.sh rather than a silent skip.

Docker/Docker Compose/SSH/SCP are not installed by `install-deps` (Docker needs a daemon; Homebrew's `openssh` is keg-only), so install those yourself.

---

15. Troubleshooting

"… not managed by docker control plugin". The directory lacks `.managed-by-docker-control`. Run `docker-control init`, or use `--dir` to point at the right project.

"No deployment configuration found (.deploy.json)". Run `docker-control add-deploy-config` to create one.

"No tags found in repository" on deploy. Cut a release first with `docker-control release`.

Composer can't reach private repos during release/deploy. The SSH agent forwarding daemon may not be running. Check that `ssh-agent` is up and `SSH_AUTH_SOCK` is set, then `docker-control --restart-ssh-agent`.

`trust-ca` reports no CA certificate found. The CA is only generated when the self-sign companion signs the first domain. Start the ingress **and** at least one project (`docker-control start-ingress` then `docker-control start`) before running `trust-ca`.

Browser still warns about the dev HTTPS cert. Ensure the ingress and a project are running so the CA exists, run `docker-control trust-ca`, then restart the browser. On Linux, `certutil` (Homebrew `nss`) is needed for browser trust.

`setacl` says containers aren't running. Start the project first with `docker-control start`.

Composer can't write its home (`/var/www/.composer` or `.config`). The container ACL missed directories created after startup. Run `docker-control doctor` to see the offending paths, then `docker-control doctor`

`--fix` to create the Composer/XDG homes and re-apply the ACLs. If `doctor --fix` reports `setfacl` is missing from the php image, refresh it with `docker-control pull`.

`update` refuses to run. In a non-interactive shell it won't rewrite a project without `--yes`. Remember `update` rewrites the *project* from the template (a backup is created); to upgrade the *tool*, use `upgrade`.

Too many `backup_*` folders after updates. Clean them with `docker-control cleanup-backups` (`--dry-run` first to preview).

Docker image version warnings on start. `docker-control` checks image freshness weekly and offers to pull. Set `DOCKER_CONTROL_SKIP_IMAGE_CHECK=1` to disable.

This manual documents `docker-control` and is kept alongside the source. For a concise command list, run `docker-control --help`.